

¿Cuáles son los tipos de Datos en Python?

Los tipos de datos integrados en Python son:

- Numéricos: int, float, complex
- Texto: str
- Secuencias: list, tuple, range
- Mapeos: dict
- Conjuntos: set, frozenset
- Booleanos: bool (True/False)
- Binarios: bytes, bytearray, memoryview
- Nulo: NoneType (valor None)

El lenguaje Python opera bajo un modelo de tipado dinámico y fuerte, esto significa que no es necesario declarar el tipo de una variable al crearla; Python deduce automáticamente en tiempo de ejecución y evita conversiones implícitas entre tipos incompatibles, garantizando mayor seguridad sin sacrificar flexibilidad.

¿Qué tipo de convención de nomenclatura deberíamos utilizar para las variables en Python?

Las variables en Python deben nombrarse usando snake case: letras minúsculas separadas por guiones bajos (por ejemplo, usuario activo o lista precios). Esta misma convención aplica a funciones y métodos. Las constantes se escriben en mayúsculas con guiones bajos (MAX_INTENTOS = 3).

Es recomendable evitar nombres de una sola letra (excepto en contadores de bucles), mezclar estilos o usar abreviaturas poco claras. Por convención, un guion bajo inicial (_variable) señala que el elemento es de uso interno o privado del módulo, aunque Python no impone restricciones estrictas de acceso.

¿Qué es un Heredoc en Python?

Un heredoc es una sintaxis para definir cadenas multilínea que respetan saltos de línea y pueden incluir variables. En Python no existe una construcción llamada "heredoc" de forma nativa. Internamente, Python almacena estas cadenas como objetos str estándar que conservan los caracteres \n y el formato visual.

Python logra el mismo objetivo de forma nativa mediante cadenas encerradas en triples comillas simples o dobles ("""...""" o '''...''').

Un ejemplo:

```
texto = """Esta es una cadena
que ocupa varias líneas
y no necesita escapes para saltos de línea."""
```

¿Qué es una interpolación de cadenas?

La interpolación de cadenas es la técnica que permite insertar valores de variables o expresiones dentro de una cadena de texto de forma dinámica. En Python hay varias formas:

- **f-strings**: la más moderna y recomendada.

```
nombre = "Ana"
edad = 30
print(f"{nombre} tiene {edad} años")
```

- **Método format()**: más antiguo pero muy versátil.

```
print("{} tiene {} años".format(nombre, edad))
```

- **Operador %** (estilo C, obsoleto para nuevo código)

```
print("%s tiene %d años" % (nombre, edad))
```

- **Template strings** (módulo string): usado cuando la cadena proviene de fuentes no confiables.

¿Cuándo deberíamos usar comentarios en Python?

Los comentarios deben usarse para explicar el "por qué", no el "qué".

Algunos casos apropiados:

- Explicar la lógica compleja o poco obvia.
- Aclarar decisiones de diseño o advertencias sobre efectos secundarios.
- Documentar el propósito de una función, clase o módulo.
- Marcar tareas pendientes (# TODO: ...) o correcciones (# FIXME: ...).

- Dejar comentarios temporales durante la depuración (pero se deben eliminar antes de producción).

No se deben usar para:

- Decir lo obvio.
- Sustituir un código que podría escribirse de forma más clara.
- Comentar código sin una razón clara.

¿Cuáles son las diferencias entre aplicaciones monolíticas y de microservicios?

Una aplicación monolítica concentra toda la lógica, las interfaces y la base de datos en un único proyecto y unidad de despliegue. Esto simplifica el desarrollo inicial, las pruebas locales y la operación, pero puede volverse difícil de escalar, mantener o actualizar de forma ágil a medida que el equipo y la base de código crecen.

La de microservicios fragmenta el sistema en componentes independientes, cada uno con su propio ciclo de vida, despliegue aislado y por lo general con su propia base de datos. Esto permite escalar selectivamente, desplegar sin afectar al resto y contener fallos, aunque exige una infraestructura más sólida y mayor coordinación organizativa para gestionar la complejidad añadida.